

AquaLLM – Project Summary for Presentation

a. Introduction (1 slide)

We have built a virtual pond to simulate the aquaculture of Largemouth Bass. It is a simplification with some objects: fish, dead fish, fish's fecal, food, probiotic, oxygen, NH3 bubbles, Disease Zone, Parasite Zone, and Pollutants. They have cyclic interaction to mimic the realistic environment.

AquaLLM is an LLM-based agent system designed to optimize aquaculture pond management. The goal is to find the best daily policies for **food**, **probiotic**, and **oxygen** inputs to maximize a composite fitness score:

$$\text{Fitness} = 0.55 \times \text{Yield} + 0.33 \times \text{SavingRate} + 0.12 \times \text{Healthiness}$$

Rather than using traditional optimization methods (e.g., genetic algorithms, reinforcement learning), we explore whether a large language model can reason about observations from a simulated pond environment and iteratively improve management policies.

We have demo simulation by running the virtual pond with the optimized policies from the LLM at the end.

b. Previous Work (1 slide)

- **Traditional aquaculture optimization:** Rule-based and model-predictive control approaches for feeding schedules and water quality management.
 - **Reinforcement learning for resource management:** RL agents trained on simulated environments to learn optimal policies (e.g., feeding, aeration).
 - **LLM-based agents (ReAct, Toolformer, Plan-and-Solve):** Recent work showing LLMs can act as decision-making agents by combining reasoning, tool use, and planning.
 - **Gap:** Limited exploration of using LLMs as policy optimizers in domain-specific simulation environments like aquaculture.
-

c. Technical Approach (2 slides)

Slide 1 – Modular Agent Architecture:

The LLM agent operates with four toggleable modules:

Module	Role in AquaLLM
Observation (Obs)	Reads pond state – water quality, shrimp health, current yield
Tool (Tool)	Executes actions in the pond simulator (set food/probiotic/oxygen levels)
Planning (Plan)	Generates a multi-step strategy for the growth cycle
Memory (Mem)	Retains results from previous runs to inform future policy adjustments

Different configurations (e.g., no_memory = Obs+Tool+Plan) allow ablation studies to measure each module's contribution.

Slide 2 – Optimization Loop:

1. The LLM **observes** the current pond state.
2. It **plans** a policy – daily food, probiotic, and oxygen levels.
3. The policy is **executed** via tool calls to the pond simulator.
4. The fitness score is computed and **saved to memory**.
5. On the next run, the LLM reviews past results and **adjusts** the policy to improve fitness.

This creates an iterative optimization cycle where the LLM learns from its own history without gradient-based training.

d. Implementation (1 slide)

- **Model:** Local Ollama qwen2.5:7b
- **Language:** Python (run_llm.py as the main entry point)
- **Pond Simulator:** A virtual pond environment that simulates shrimp growth, water quality, and resource costs over a growth cycle. *(Brief mention – not the focus.)*

- **LLM Agent:** Runs in Google Colab with H100 GPU (/content/AquaLLM/), with defined configs to toggle modules on/off.
- **Logging:** Execution traces saved via `_save_trace()`; memory persisted in `memory_full.json` for cross-run learning.
- **Configs tested:** `full` (all modules), `no_memory` (Obs+Tool+Plan), and others for ablation.

e. Data (1 slide)

- **Stochastic and dynamic environment:** The "data" comes from the pond simulator — each run produces a full growth cycle of observations and outcomes.
- **Inputs:** Daily policy vectors (food amount, probiotic dose, oxygen level).
- **Outputs:** Yield (harvest weight), SavingRate (cost efficiency), Average Healthiness (health, fullness, oxygen, and immunity).
- **Evaluation:** Multiple runs per config, each producing a fitness score for comparison.

f. Results (1–2 slides)

Present the fitness scores achieved across configurations:

Config	Fitness Score	Notes
<code>full</code> (Obs+Tool+Plan+Mem)	—	Memory allows iterative improvement
<code>no_memory</code> (Obs+Tool+Plan)	—	Each run starts fresh — no learning
Other ablations	—	Isolate planning vs. tool contributions

Key observations (*fill in with your actual numbers*):

- Did memory-enabled runs improve fitness over successive iterations?
- Which component of fitness (Yield vs. SavingRate vs. Healthiness) was hardest to optimize?

- How many iterations did the LLM need to converge on a strong policy?

g. Conclusion & Future Work (1 slide)

Conclusion:

- AquaLLM demonstrates that an LLM agent can serve as a policy optimizer for aquaculture management by combining observation, tool use, planning, and memory.
- The modular design enables clear ablation analysis of which cognitive capabilities matter most for this domain.

Future Work:

- Test with more complex pond simulators (multiple species, variable weather).
- Compare LLM-optimized policies against RL baselines and expert heuristics.
- Explore different LLM backbones and prompt strategies.
- Extend memory to support longer optimization horizons and transfer across different pond configurations.